

Algorithmen und Datenstrukturen



Warum sortieren wir?

Warum sortieren wir?

Durchsuchen einer unsortierten Menge

Um die Anwesenheit eines Werts / Objekts in einer unsortierten Menge festzustellen bzw. auf dessen Attribute zugreifen zu können, müssen wir ggf. die gesamte Menge durchlaufen, im Schnitt aber die halbe Menge (manchmal werden wir schnell fündig, manchmal nicht). Damit ergibt sich:

Laufzeit im Durchschnitt $O(n)$

Warum sortieren wir?

Durchsuchen einer sortierten Menge

BINSEARCH(Z, s, l, r)

```
1: if  $l > r$  then                                ▷  $l$  hat  $r$  überholt
2:   return -1                                       ▷  $s$  nicht in  $Z$  enthalten
3: end if
4:  $m \leftarrow (l + r) / 2$                             ▷ Mitte des aktuellen Bereichs
5: if  $s < Z[m]$  then
6:   return BINSEARCH( $Z, s, l, m - 1$ )
7: else if  $s > Z[m]$  then
8:   return BINSEARCH( $Z, s, m + 1, r$ )
9: else
10:  return  $m$ 
11: end if
```

Bei einer sortierten Menge kann durch eine binäre Suche die Laufzeit auf $O(\log n)$ reduziert werden.

M.a.W.: Sortieren lohnt sich, wenn viele Suchvorgänge erwartet werden und der Aufwand für das Sortieren durch Einsparungen beim Suchen aufgewogen wird.

Binärer Suchbaum

Binärer Suchbaum

Grundidee

Statt es überhaupt erst zu einer (großen) Menge unsortierter Elemente in einem Feld/Array/Vektor kommen zu lassen, wird bereits beim Zusammenstellen der Menge eine besser geeignete Datenstruktur verwendet, effizientes Suchen und Sortieren erlaubt.

Die Datenstruktur soll darüber hinaus dynamisch wachsen können, weil die Anzahl der Elemente a priori nicht bekannt ist.

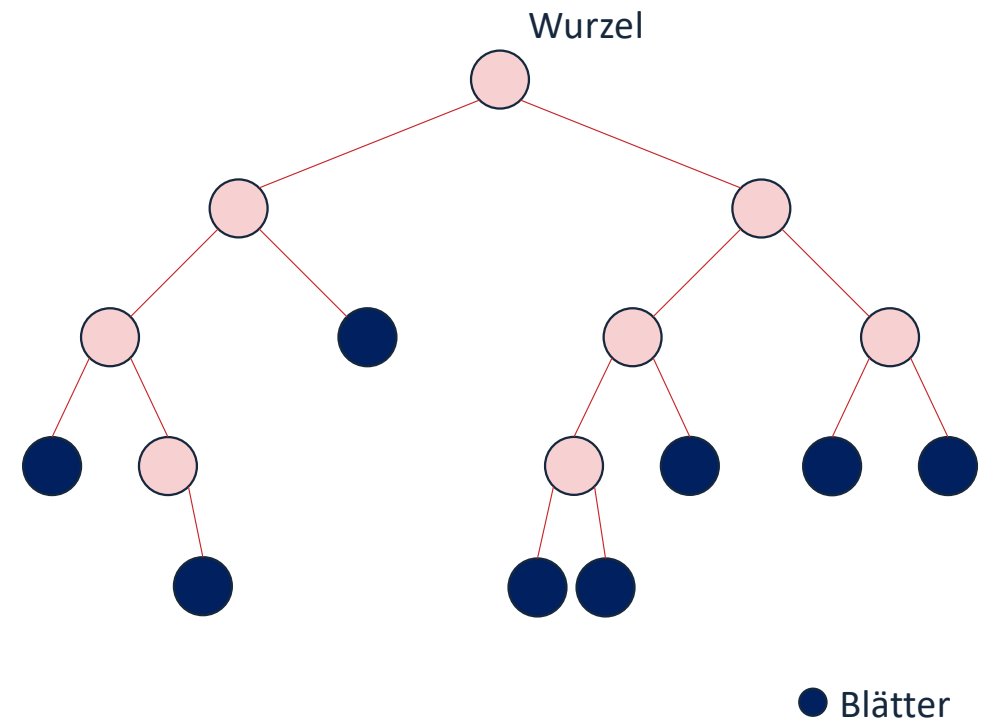
Eine Möglichkeit für eine solche Datenstruktur ist der **binäre Suchbaum**.

Binärer Suchbaum

Binärer Baum

Ein binärer Baum

- besteht aus Knoten mit max. zwei (daher binär) Nachfolgern/Kindern
- Mit Ausnahme der Wurzel besitzt jeder Knoten genau einen Elternknoten
- Knoten ohne Kinder heißen äußere Knoten (oder auch Blätter)
- Anders als beim Heap, bei dem sich der Baum gleichmäßig zeilenweise füllt, kann ein binärer Baum auch sehr unbalanciert sein.
- Ein einfaches Abbilden auf ein Array ist daher nicht möglich
- Typischerweise werden binäre Bäume durch verkettete Elemente auf dem Heap gebildet.



Binärer Suchbaum

Binärer Suchbaum

Ein binärer Baum wird zum binären Suchbaum, wenn die Knoten vergleichbare Werte (Keys) besitzen und gilt:

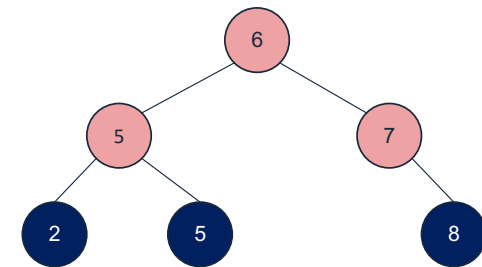
Sei x ist ein Knotenwert in Baum $\Rightarrow$

$\forall v$ im linken Teilbaum: $v \leq x$

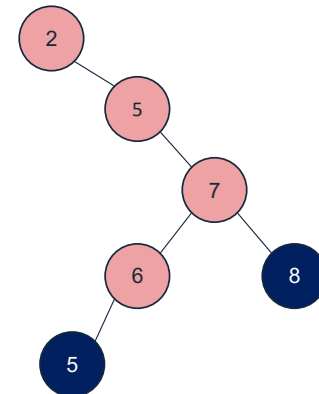
$\forall w$ im rechten Teilbaum: $x \leq w$

Die beiden Beispiele auf der rechten Seite repräsentieren die gleichen Elementmengen, d.h. ein binärer Suchbaum zu einer Menge ist nicht eindeutig.

Beispiel 1



Beispiel 2



Binärer Suchbaum

Sortieren

Die sortierte Ausgabe wird auch

Inorder-Traversierung bzw. **Inorder-Tree-Walk** genannt.

„Inorder“, weil die Verarbeitung des aktuellen Werts zwischen der Verarbeitung des linken und des rechten Teilbaums stattfindet.

Der nebenstehende Algorithmus wird hierzu initial mit dem Wurzelknoten als Parameter aufgerufen.

INORDER-TREE-WALK(x)

```
1: if  $x \neq \text{NIL}$  then  
2:   INORDER-TREE-WALK( $x.\text{left}$ )  
3:   print  $x.\text{key}$   
4:   INORDER-TREE-WALK( $x.\text{right}$ )  
5: end if
```

Jeder Knoten wird durch eine Struktur repräsentiert mit den Attributen:

key: Schlüsselwert des Knoten

left: Verweis zum linken Nachfolger (oder NIL)

right: Verweis zum rechten Nachfolger (oder NIL)

Effizienzbetrachtung

Jeder Knoten wird genau einmal besucht, daher Laufzeit: $O(n)$

Binärer Suchbaum

Suchen

Gesucht wird der Knoten im Baum, der den angegebenen Schlüsselwert besitzt. Existiert ein solcher Knoten, wird eine Referenz auf ihn zurückgegeben, sonst NIL

Der nebenstehende Algorithmus wird hierzu initial mit der Wurzel des Baums als Startknoten aufgerufen.

Effizienzbetrachtung

Laufzeit: $O(h)$, wobei h die Höhe des Baums ist

TREE-SEARCH(*start*, *key*)

```
1: if start = NIL or key = start.key then  
2:   return start  
3: end if  
4: if key < start.key then  
5:   return TREE-SEARCH(start.left, key)  
6: else  
7:   return TREE-SEARCH(start.right, key)  
8: end if
```

Binärer Suchbaum

Einfügen eines Elements

Sei z der einzufügende Knoten und T der betroffene Baum.

Ausgehend vom Wurzelknoten wird durch einen Abstieg ein NIL gesucht, das durch z ersetzt werden kann.

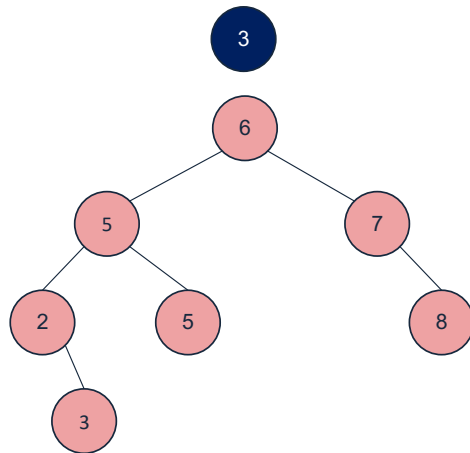
y verweist dann auf den vorgelagerten äußeren Knoten, in dem z verankert werden muss.

TREE-INSERT(T, z)

```
1:  $y \leftarrow \text{NIL}$ 
2:  $x \leftarrow T.\text{root}$ 
3: while  $x \neq \text{NIL}$  do
4:    $y \leftarrow x$ 
5:   if  $z.\text{key} < x.\text{key}$  then
6:      $x \leftarrow x.\text{left}$ 
7:   else
8:      $x \leftarrow x.\text{right}$ 
9:   end if
10: end while
11: if  $y = \text{NIL}$  then
12:    $T.\text{root} \leftarrow z$ 
13: else if  $z.\text{key} < y.\text{key}$  then
14:    $y.\text{left} \leftarrow z$ 
15: else
16:    $y.\text{right} \leftarrow z$ 
17: end if
```

Binärer Suchbaum

Einfügen eines Elements



Effizienzbetrachtung

Laufzeit: $O(h)$, wobei h die Höhe des Baums ist

TREE-INSERT(T, z)

```

1:  $y \leftarrow \text{NIL}$ 
2:  $x \leftarrow T.\text{root}$ 
3: while  $x \neq \text{NIL}$  do
4:    $y \leftarrow x$ 
5:   if  $z.\text{key} < x.\text{key}$  then
6:      $x \leftarrow x.\text{left}$ 
7:   else
8:      $x \leftarrow x.\text{right}$ 
9:   end if
10: end while
11: if  $y = \text{NIL}$  then
12:    $T.\text{root} \leftarrow z$ 
13: else if  $z.\text{key} < y.\text{key}$  then
14:    $y.\text{left} \leftarrow z$ 
15: else
16:    $y.\text{right} \leftarrow z$ 
17: end if

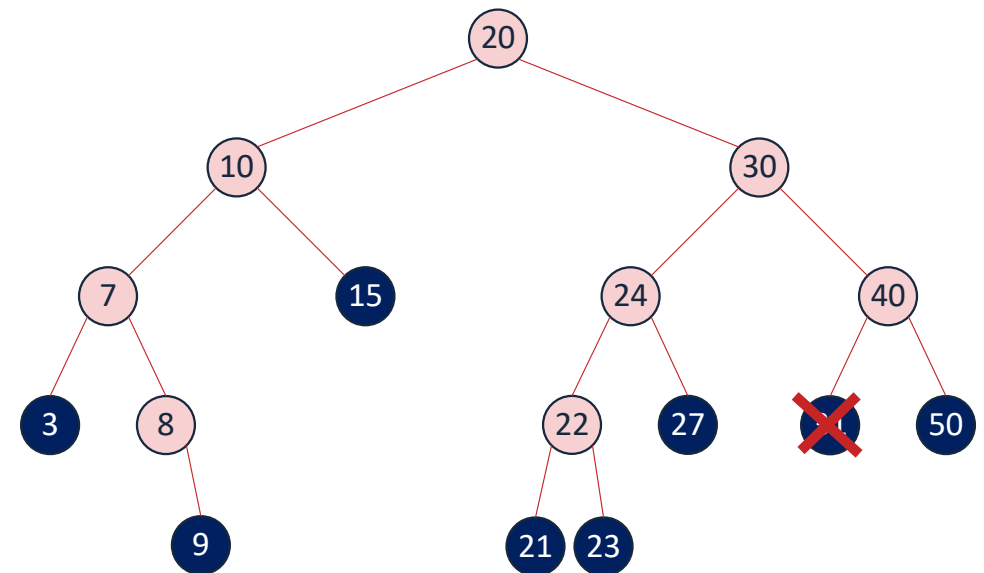
```

Binärer Suchbaum

Löschen eines Elements

1. Fall: Entfernen eines Blatts

Ein Knoten ohne Nachfolger kann einfach entfernt werden, ohne dass die Struktur des verbleibenden Binärbaums beeinflusst wird.

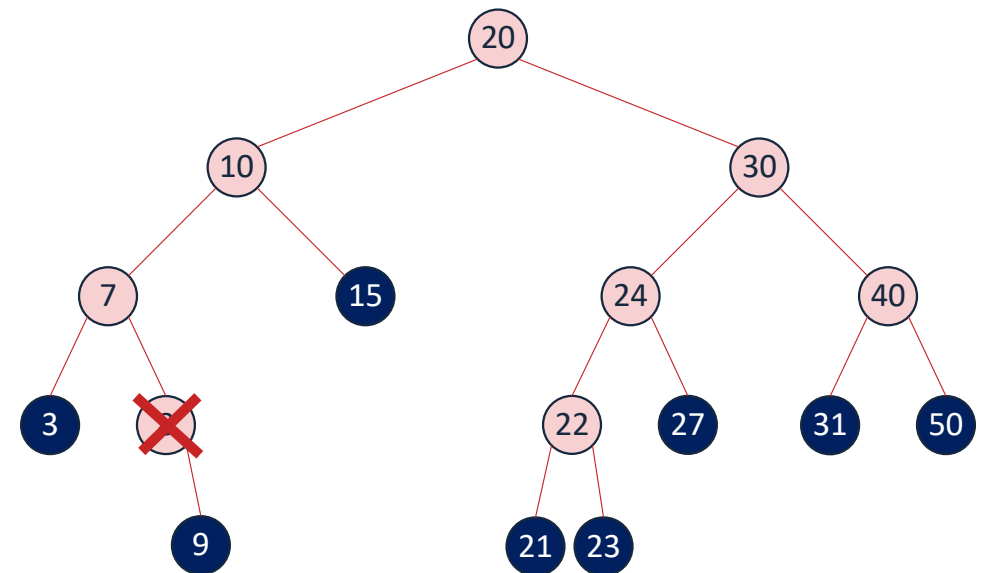


Binärer Suchbaum

Löschen eines Elements

2. Fall: Entfernen eines Knotens mit nur einem Nachfolger

Der Zeiger auf den zu entfernenden Knoten wird auf den nicht leeren Nachfolger-Knoten umgelenkt.



Binärer Suchbaum

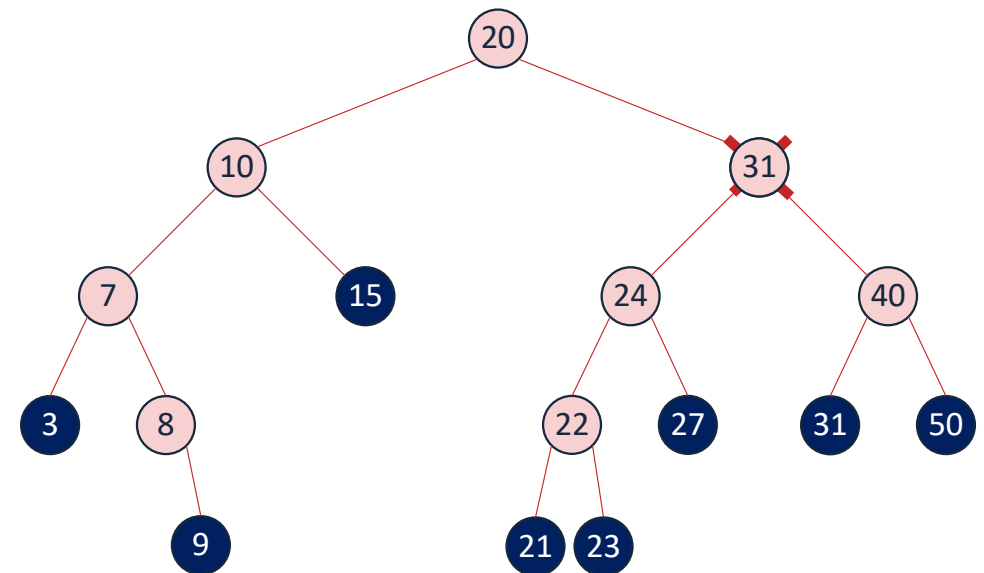
Löschen eines Elements

3. Fall: Entfernen eines Knotens mit zwei Nachfolgern

In diesem Fall muss der zu löschende Knoten durch seinen Nachfolger ersetzt werden, der sich im rechten Teilbaum befindet. Im einfachsten Fall ist dies ein äußerer Knoten.

Falls es sich beim Nachfolger um einen inneren Knoten handelt, kann dieser kein linkes Kind besitzen (sonst wäre das der Nachfolger des zu löschenden Knoten).

Wir entfernen den gefundenen Nachfolger aus dem rechten Teilbaum, indem wir das rechte Kind nach oben rücken lassen, und setzen den Nachfolger an die Stelle des zu löschenden Knoten.



Binärer Suchbaum

Laufzeit-Komplexität der Operationen

Operation	Laufzeit
Traversieren	$O(n)$
Suchen	$O(h)$
Einfügen	$O(h)$
Löschen	$O(h)$

Vielen Dank für die Aufmerksamkeit!